

Aerial Plane Attack

Architecture and code study guide

A compact guide to the project's purpose, major runtime flow, AI families, persistence, limitations, and the code path you chose to learn:

CODE FOCUS**SimulationManager -> ITrainingParadigm -> IEvolutionEngine**

Know who owns what, why the boundaries exist, and how the flow differs for evolution and reinforcement learning.

How to use this	Time
Read pages 2-5 for the project story and architecture.	10 min
Study pages 6-9 for the selected code path.	12-15 min
Skim pages 10-12 for persistence, risks, and grader questions.	8-10 min

Source basis. Both supplied architecture reports were used: the concise report for presentation priorities and the longer report for technical depth and risks. The supplied Mermaid UML was used for relationships. The current C# under Assets/Scripts was treated as authoritative when it differed or contained newer work.

Prepared from the current workspace on 25 July 2026

READ FIRST

1. The project in one page

60-SECOND PITCH

Aerial Plane Attack is a Unity 6 flight-learning simulator. It lets four AI types - Fixed NeuroEvo, NEAT, PPO, and SAC - control the same jet physics on the same tasks. The architecture separates the shared simulation from the learning lifecycle, so the algorithms can be compared fairly without duplicating the aircraft or objectives.

The five facts to remember

- **Two implemented training objectives:** Max Altitude and Flight School. Dogfight has cameras, selection, and weapons, but no combat objective or combat sensor.
- **One shared environment:** sensors produce observations; a controller produces pitch, roll, yaw, and throttle; JetPhysics moves the same Rigidbody aircraft; an objective scores and terminates the run.
- **Two training families:** evolutionary modes run their brains and evolution inside Unity; PPO/SAC use Unity ML-Agents plus an external Python trainer.
- **The key abstraction:** SimulationManager owns composition; a paradigm owns the training lifecycle; an evolution engine owns population evolution. RL has no IEvolutionEngine - its lower layer is ML-Agents/Python.
- **Experiment continuity:** settings and training saves are keyed by (*track scene, AI type*), so two Flight School scenes do not overwrite each other.

A strong opening answer

"The project is mainly an experiment platform, not just a flying game. Its most important design decision is that all AI methods share the same sensors, actions, physics, and objective contracts. SimulationManager composes the run, ITrainingParadigm hides the very different generation and episode lifecycles, and IEvolutionEngine further separates the two evolutionary algorithms. That gives us fair comparisons and clear extension points."

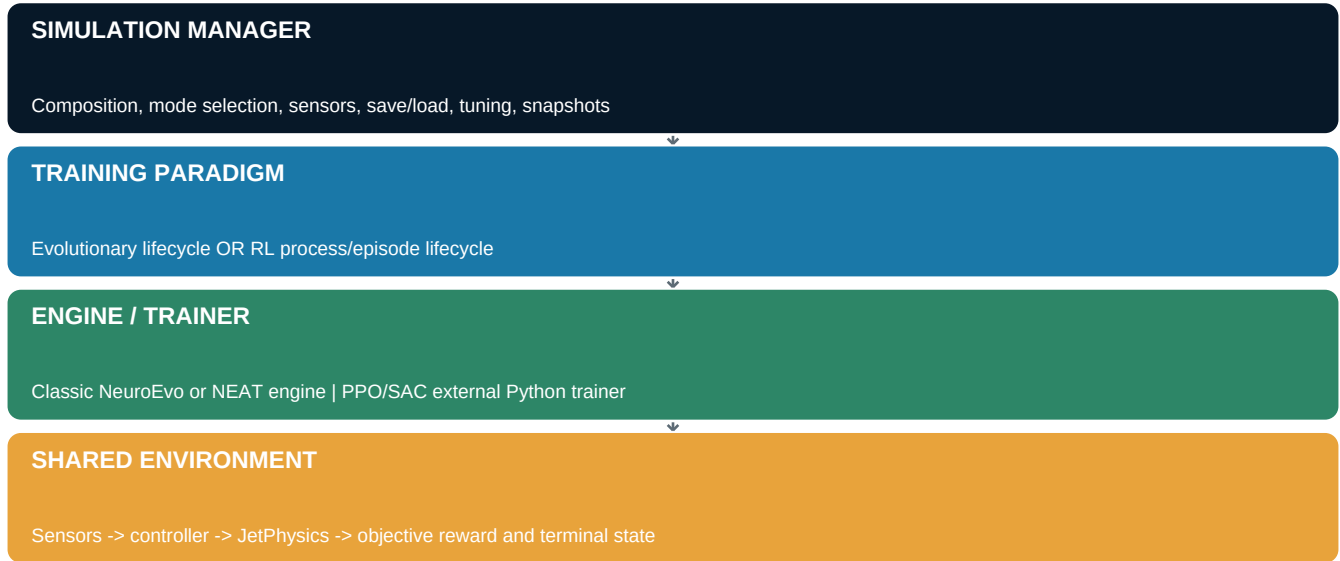
What is actually delivered

Capability	Status	Interview wording
Max Altitude	Implemented	Continuous-control baseline: gain height with low control effort.
Flight School	Implemented	Navigate ordered hoops with dense progress and alignment rewards.
Fixed NeuroEvo / NEAT	Implemented	Population evolution happens inside Unity.
PPO / SAC	Implemented with dependency	Policy learning lives in an external Python ML-Agents trainer.
Saved-policy challenge	Implemented for Flight School	Human races a frozen saved AI; no learning or saving.
Dogfight training	Incomplete	Supporting gameplay exists, but the learning objective and combat observations do not.

CORE ARCHITECTURE

2. How the system is divided

Think of the project as four stacked responsibilities. The three code layers you selected sit above a shared environment. The layer names describe ownership, not Unity folders.



The responsibility test

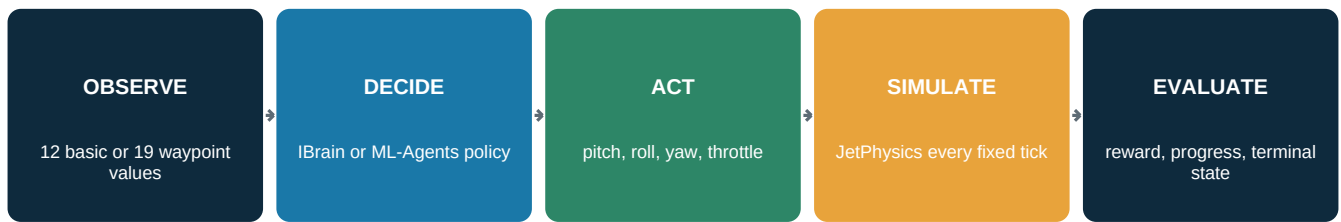
Layer	Owns	Does not own
SimulationManager	Composition, population creation, sensor activation, mode switching, manager-level save/load, tuning routes, UI snapshot fields.	Selection/mutation math or reward formulas.
ITrainingParadigm implementation	The lifecycle: generation boundaries for evolution; trainer/Academy and checkpoint lifecycle for RL; paradigm save/inference behavior.	Aircraft aerodynamics or objective definitions.
IEvolutionEngine implementation	Brain creation, selection/speciation, mutation/evolution, champion, and opaque population serialization.	Jets, Unity scene objects, physics, or objectives.
Shared environment	Observation vectors, controls, physics, spawn rules, rewards, progress, and terminal conditions.	Which learning algorithm is active.
CRITICAL NUANCE The three-layer chain is exact for Fixed NeuroEvo and NEAT. For PPO/SAC the chain is SimulationManager -> RLParadigm -> ML-Agents/Python trainer . Saying that RL uses an IEvolutionEngine would be incorrect.		

Important supporting contracts

- **IObjective:** chooses the sensor, spawns jets, returns step reward and final fitness, checks terminal state, exposes reward breakdown and tunable parameters.
- **ISensor:** returns a fixed-width observation array and its count.
- **IBrain:** in-process callable policy used by evolutionary jets. PPO/SAC deliberately do not expose an in-process brain.
- **SimulationSnapshot:** stable read model that keeps telemetry from depending directly on a concrete paradigm.

RUNTIME STORY

3. The shared observe-decide-act loop



Why the comparison is meaningful

All four AI types receive observations from the same active sensor, drive the same four flight controls, and are judged by the same objective against the same JetPhysics model. What changes is the learning mechanism and its lifecycle. SimulationManager also stamps the active sensor count into the selected model settings, preventing a stale configured input width.

Observations and actions

Item	What it contains	Count / range
BasicFlightSensors	Local velocity, local angular velocity, forward vector, up vector.	12 normalized floats
WaypointSensors	All basic values plus local target direction, normalized distance, and hoop forward direction.	19 floats
Flight actions	Pitch, roll, yaw, throttle. Throttle is remapped from [-1, 1] to [0, 1].	4 continuous outputs
Optional weapon actions	Fire and switch weapon.	Outputs 5-6; not scored by an implemented objective

The objectives

Objective	Reward idea	Terminal conditions	Sensor
Max Altitude	Vertical gain minus lambda times added control effort. Final fitness is total height gain minus total effort penalty.	Crash or maximum time.	BasicFlight
Flight School	Progress toward hoop, forward alignment, hoop-pass bonus, look-direction penalty, effort penalty, drift penalty, and completion time bonus.	Crash, total time, too long since last hoop, course complete, or crossing a hoop plane outside the ring.	Waypoint
SUBTLE BEHAVIOR Flight School advances the next hoop and retargets WaypointSensors inside GetStepReward(). That is why evolutionary inference still calls GetStepReward even though it does not learn from the returned value.			

ALGORITHM MAP

4. What changes between the four AI types

AI type	Policy representation	Training boundary	Where learning happens
Fixed NeuroEvo	Dense feed-forward NeuroEvoBrain with fixed layer shape.	Wait for the entire population, then evolve a generation.	Inside Unity: ClassicNeuroEvoEngine.
NEAT	SharpNEAT genomes that evolve weights and topology, decoded to NeatBrain.	Wait for the population, stamp fitness, step SharpNEAT one generation.	Inside Unity: NeatEngine + SharpNEAT.
PPO	ML-Agents policy and optimizer state.	Agents end episodes independently; trainer updates a shared policy.	External Python process.
SAC	ML-Agents off-policy actor/critic state and replay behavior.	Independent episodes and trainer updates.	External Python process.

Evolutionary family

- **Synchronous batch:** a terminal jet is finalized and deactivated; evolution waits until aliveCount reaches zero.
- **Fixed topology:** top performers survive through elitism; remaining brains come from tournament selection and mutation.
- **NEAT:** SharpNEAT handles species and structural evolution. Negative project scores are shifted upward because SharpNEAT fitness must be nonnegative.

Reinforcement-learning family

- **Independent episodes:** JetMLAgent receives an action, applies it, asks the objective for reward, calls AddReward, and ends its own episode when terminal.
- **External ownership:** RLParadigm writes YAML, launches mlagents-learn, binds BehaviorParameters and DecisionRequester, and coordinates process shutdown/resume.
- **No IBrain object:** GetChampionBrain returns null for PPO/SAC because the live policy is not a callable C# object. The UI can see observations and last actions, not trainer-side weights.

The fairness claim - and its limit

WHAT YOU CAN CLAIM

The environment interface is shared, so comparisons control many implementation variables. However, population generations and RL episodes are different sampling/update regimes, so equal wall-clock time or equal episode count is not automatically a perfectly fair experimental budget.

A useful contrast answer

Why not one giant AI class? Because generation-based evolution and asynchronous RL episodes do not share a natural loop. The common interface standardizes application lifecycle - initialize, tick, snapshot, persistence, inference, dispose - while allowing each implementation to remain algorithmically correct.

CODE DEEP DIVE 1 OF 3

5. SimulationManager: composition and routing

MEMORY SENTENCE

The manager decides what exists and where calls go; it does not perform learning.

Startup sequence in Start()

1. Cast the scene's objectiveProvider to IObjective and stop if the scene is misconfigured.
2. Load settings for the active track; optionally replace them with defaults for the AI type selected in the menu.
3. Instantiate the population once, enable exactly the objective's required sensor, and derive the model input size from that sensor.
4. Create reward, hyperparameter, and network-shape tuning services.
5. Map AIType to the correct paradigm and initialize it with population, settings, and objective.
6. Optionally load a saved run or enter the saved-policy challenge requested by the menu.

Managers/SimulationManager.cs - the central tick router

```
private void FixedUpdate()
{
    if (inChallengeMode) { TickChallenge(); return; }
    if (inInferenceMode)
    {
        activeParadigm?.TickInference();
        return;
    }
    activeParadigm?.Tick();
}
```

Managers/SimulationManager.cs - AI type to lifecycle

```
private ITrainingParadigm CreateParadigm(AIType type)
{
    switch (type)
    {
        case AIType.FixedNeuroEvo:
            return new EvolutionaryParadigm(new ClassicNeuroEvoEngine());
        case AIType.NEAT:
            return new EvolutionaryParadigm(new NeatEngine());
        case AIType.PPO_MLAgents:
        case AIType.SAC_MLAgents:
            return new RLParadigm();
        default:
            return null;
    }
}
```

What to point out in the code

- This is dependency injection by constructor for evolution: the same EvolutionaryParadigm receives either engine.
- PPO and SAC share RLParadigm; the selected AIType changes generated YAML and algorithm-specific settings.
- The manager has explicit modes for training, inference, and challenge, so exactly one lifecycle is pumped per FixedUpdate.

CODE DEEP DIVE 2 OF 3

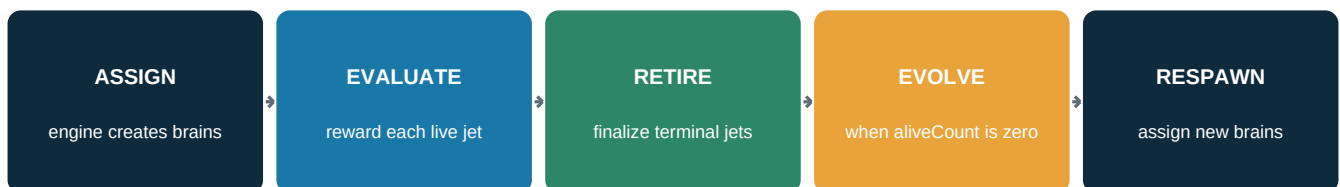
6. ITrainingParadigm: hide lifecycle differences

The interface is an application-facing strategy. SimulationManager can initialize, tick, read telemetry, save/load, enter inference, and dispose without knowing whether the algorithm runs batches in C# or episodes through Python.

Managers/Paradigms/ITrainingParadigm.cs - reduced to the methods to remember

```
void Initialize(List<JetAgent> population,
               SimulationSettings settings,
               IObjective objective);
void Tick();
SimulationSnapshot GetSnapshot();
void SaveState();
void LoadState();
bool CanRunInference { get; }
bool StartInference();
void TickInference();
void Dispose();
```

EvolutionaryParadigm owns a generation



- During evaluation: `CurrentFitness += GetStepReward()`. On terminal: `CalculateTotalFitness()`, deactivate, decrement `aliveCount`.
- At the generation boundary: gather scores, call `engine.EvolveNextGeneration()`, update statistics, assign brains, reset and respawn all jets.

RLParadigm owns the trainer lifecycle

- Initialize deliberately does not start Python. The first training Tick starts fresh; LoadState can instead stage a checkpoint and start directly in resume mode.
- BehaviorParameters are configured before adding/enabling JetMLAgent, preventing registration with a zero-action default behavior.
- JetMLAgent, not RLParadigm.Tick, performs the per-action observe/reward/terminal loop. RLParadigm owns process setup, statistics, persistence, Academy recycling, and cleanup.

GOOD DESIGN EXPLANATION

The interface does not pretend the loops are identical. It standardizes only what the rest of the application genuinely needs. That is a Strategy pattern at the training-lifecycle level.

CODE DEEP DIVE 3 OF 3

7. IEvolutionEngine: isolate population algorithms

Managers/Engines/IEvolutionEngine.cs - the engine boundary

```
List<IEvolvableBrain> InitializeGeneration(SimulationSettings settings);
List<IEvolvableBrain> EvolveNextGeneration(List<float> fitnessScores);
int GetLastGenerationBestEliteIndex();
IEvolvableBrain GetChampionBrain();
float GetChampionScore();
string CaptureState();
List<IEvolvableBrain> RestoreState(
    string stateJson, SimulationSettings settings);
```

ClassicNeuroEvoEngine

- Creates one dense NeuroEvoBrain per population slot using the configured network shape.
- Sorts brain/score pairs descending, updates an all-time champion, and preserves at least the top 1% as elites.
- Uses tournament selection of five candidates for every non-elite child, then copies and mutates the winner.
- Reads parents from an immutable parentPool snapshot so earlier overwritten child slots cannot accidentally become parents.
- CaptureState serializes shape, every flattened brain, champion, champion score, and generation as JSON.

NeatEngine

- Builds SharpNEAT's genome factory, decoder, speciation strategy, and manually steppable evolution algorithm.
- Stamps scores onto the current genome objects before StepOneGeneration, because SharpNEAT rebuilds/reorders its genome list internally.
- Shifts negative scores and adds a baseline so all SharpNEAT fitness values are positive.
- Guards the top 2% of genomes against probabilistic elite loss, then locates the previous winner's new slot for the spectator camera.
- CaptureState stores complete population and champion genome XML inside an opaque JSON payload.

WHY OPAQUE STATE MATTERS

EvolutionaryParadigm does not know whether a brain is flattened neural weights or a SharpNEAT genome. It only stores the EngineState string. This keeps serialization details inside the engine that owns them.

Where the abstraction is imperfect

IEvolvableBrain exposes Copy and Mutate, which fits fixed topology but not NEAT, where those operations belong to the genome/evolution engine. A cleaner future split would separate inference-capable phenotype from directly mutable fixed-topology brain.

CONNECT THE CODE

8. Two end-to-end traces

Trace A: one Fixed NeuroEvo generation

1. SimulationManager creates EvolutionaryParadigm(new ClassicNeuroEvoEngine()).
2. Initialize asks the engine for brains, assigns one to each JetAgent, configures decision cadence, and calls objective.SetStartingState.
3. JetAgent periodically reads its active ISensor, calls Brain.GetControlOutputs, and reapplies the cached action between decision ticks.
4. JetPhysics integrates thrust, lift, drag, control torque, damping, and stability each physics step.
5. EvolutionaryParadigm accumulates GetStepReward. Terminal jets receive CalculateTotalFitness and are deactivated.
6. When all jets are done, the engine receives fitness scores, returns evolved brains, and the paradigm respawns the next generation.

Trace B: one PPO/SAC episode

1. SimulationManager creates RLParadigm. Its first Tick writes YAML and launches mlagents-learn unless this is a resume/inference path.
2. RLParadigm configures BehaviorParameters and DecisionRequester, then injects objective/paradigm references into JetMLAgent.
3. JetMLAgent.CollectObservations copies the active sensor vector into ML-Agents.
4. The Python policy returns continuous actions. JetMLAgent applies them to the same JetPhysics component.
5. JetMLAgent asks the same IObjective for reward, calls AddReward, and ends the episode on terminal.
6. OnEpisodeBegin reports the previous score to RLParadigm, resets the jet, and asks the objective for a fresh starting state.

**IF ASKED WHERE
LEARNING
HAPPENS**

Fixed/NEAT: Unity C# engine returns a new population.

PPO/SAC: Python updates a shared policy from trajectories sent through ML-Agents; Unity is the environment.

The simplest relationship diagram to redraw



IMPORTANT SUPPORTING BEHAVIOR

9. Save, inference, tuning, and challenge

Save identity and payload

- **Identity:** GameData/<Track>/save_<AIType>.json. Track is the sanitized active scene name; objective mode still selects baked defaults and the active sensor.
- **Common payload:** cloned SimulationSettings, objective parameters, generation/episode number, population and score statistics, elapsed time/history, and an opaque EngineState marker/blob.

Path	What is persisted	Resume behavior
Fixed NeuroEvo	All flattened brains plus champion in JSON.	Engine restores brains; paradigm reassigns and respawns them.
NEAT	Complete population and champion genome XML inside JSON.	SharpNEAT scaffolding is rebuilt, genomes are read, IDs reseeded, and evolution resumes.
PPO / SAC	Trainer results directory is copied to a stable per-track checkpoint slot; JSON stores metadata/marker.	Checkpoint is staged back into results and Python launches with --resume.

Inference

- **Evolution:** load the saved champion IBrain into one jet and loop the objective in process with no evolution.
- **RL:** stage the checkpoint and launch mlagents-learn --resume --inference. It is frozen, but still needs Python and the Academy connection.

Runtime tuning

- **Hot:** reward and compatible scalar changes use a protected Save -> Load round trip so trained state survives. A backup prevents the user's manual save from being overwritten.
- **Cold:** population size or network architecture makes weights structurally incompatible. Persist settings and rebuild from scratch; the manual save remains untouched.

Saved-policy challenge

The current manager also supports a Flight School-only race between a human jet and one frozen saved AI. It loads the saved settings/objective, starts inference for the AI, creates a human clone, disables weapons and mutual collisions, uses the same starting line, and ranks by hoops passed then average speed. It does not learn or write a save.

RL SAVE CAVEAT The Save button can only copy the latest trainer checkpoint. Progress since that checkpoint is not captured, so RL saves are checkpoint-bounded rather than exact snapshots of the current frame.

PRESENTATION AND OPERATION

10. A clean grader demo story

Suggested live sequence

1. **Choose a track and AI type.** Explain that the menu choice enters GameSession and the scene objective determines the sensor and reward contract.
2. **Show several jets training.** Point out that the same population and physics exist regardless of algorithm.
3. **Open telemetry.** Explain generation versus episode counts, current max/average versus all-time best, selection, and brain visualization limits for RL.
4. **Select a jet.** Connect live behavior to observations, actions, and the objective's reward breakdown.
5. **Show save/load or inference.** State the difference between full training state and one-policy replay.
6. **If stable, show tuning or challenge.** Use it to explain hot/cold compatibility or frozen-policy evaluation.

Operational points worth knowing

Topic	What to say
Unity version	Unity 6, project baseline 6000.3.10f1.
Evolutionary run	No external build step; runs fully in Unity.
RL run	Needs a compatible ML-Agents Python environment; RLParadigm generates trainer YAML every run.
Windows build	Trainer interpreter lookup prefers a bundled StreamingAssets environment, then MLAGENTS_PYTHON, then Conda.
Standalone connection	The player must run with --mlagents-port 5004; AppBootstrap relaunches the Windows player if the argument is missing.
Bundling	Thin builds download a pinned environment on first run; self-contained builds ship the extracted environment.

Do not overclaim

- Do not call Dogfight an implemented AI training mode. Say the gameplay foundation exists, but objective, combat sensor, teams, reward attribution, and terminal rules are missing.
- Do not say RL inference is embedded in Unity. It currently launches the external trainer in inference mode.
- Do not say Save captures the exact current PPO/SAC weights. It captures the latest available checkpoint.
- Do not say every algorithm has an IBrain or IEvolutionEngine. Those are in-process evolutionary concepts.

IF THEY ASK FOR CRITIQUE

11. Limitations and strongest improvement answers

Issue	Why it matters	Good improvement
Trainer startup can block	Port readiness waits can make the Editor/player look frozen.	Use an async coroutine/state machine and expose starting/ready/failed status to telemetry.
RL inference is expensive	Python, port binding, Academy startup, and the large environment are still required.	Export ONNX and run inference in Unity with Sentis for deployment.
RL saves are checkpoint-bounded	The UI can imply a precise snapshot when recent steps may be missing.	Show captured checkpoint step/age; optionally coordinate a trainer-side checkpoint request.
SimulationManager is growing	Training, persistence, tuning, inference, and challenge increase responsibility and test difficulty.	Extract challenge and run-transition services while keeping the manager as composition root.
Flight School reward has side effects	Advancing hoops in GetStepReward creates a hidden requirement for inference and future evaluators.	Separate environment progression from pure reward calculation.
Global/static coupling	Static tuners, session state, events, and singletons complicate isolated tests and parallel simulations.	Inject explicit services/references at the scene composition root.
Serialized-default drift	Inspector values override C# initializers and can silently disagree with defaults.	Centralize defaults, validate assets at startup, and add migration/tests.

Best answer to 'what would you refactor first?'

**RECOMMENDED
PRIORITY**

Make RL trainer startup asynchronous and explicit. It has direct user-visible impact, reduces apparent freezing, and creates a clean state boundary for connection failure, retries, telemetry, load, and inference. After that, embedded ONNX/Sentis inference gives the largest deployment simplification.

Best answer to 'how would you add Dogfight?'

Implement a DogfightObjective and combat sensor first, then define team/spawn semantics, damage ownership, kill/death rewards, terminal rules, and a six-action output shape. Add defaults for each supported AI type, validate save/inference, and only then expose the mode as complete. Existing weapons, selection, and camera code are presentation/gameplay support, not the missing learning contract.

Tests that would buy confidence quickly

- Settings cloning and track-keyed save paths.
- Sensor widths and action-size configuration for every objective/AI pair.
- Classic and NEAT CaptureState/RestoreState round trips.
- Hot/cold tuning classification and manual-save protection.
- Objective progression/terminal behavior, especially hoop crossing and inference progression.

FINAL REVISION

12. Likely grader questions - short answers

Question	Answer to lead with
What is the architectural center?	SimulationManager is the composition root and tick router; it delegates lifecycle behavior to ITrainingParadigm.
Why a paradigm and an engine?	Paradigm abstracts the whole training lifecycle. Engine is a lower, evolution-only abstraction for how a population is created, evolved, championed, and serialized.
How are algorithms compared fairly?	They share objective-selected observations, the four flight actions, JetPhysics, spawn rules, reward, and terminal logic.
Why both JetAgent and JetMLAgent?	JetAgent drives an in-process IBrain. JetMLAgent adapts the same jet/physics/objective to the ML-Agents communicator and external policy.
What happens every FixedUpdate?	Manager routes to challenge, inference, or training. The active lifecycle advances; jets still integrate physics through Unity's fixed timestep.
What is saved?	Settings, objective parameters, statistics/history, and engine-specific state. Evolution stores the population; RL snapshots trainer checkpoints.
What is the biggest incomplete feature?	Dogfight training: gameplay pieces exist but objective and combat observation/reward contracts do not.
What pattern is used?	Strategy for paradigms and engines, interfaces for objectives/sensors/brains, a composition root in SimulationManager, and a snapshot read model for UI.

One-pass checklist

- I can give the 60-second pitch without notes.
- I can redraw Manager -> Paradigm -> Engine/Trainer -> Environment.
- I can explain why the engine layer does not apply to PPO/SAC.
- I can trace one Fixed NeuroEvo generation and one PPO/SAC episode.
- I can explain BasicFlight (12), Waypoint (19), and the four flight outputs.
- I can explain track-keyed saves, evolution state versus RL checkpoints, and hot versus cold changes.
- I can name the honest limitations: incomplete Dogfight, external RL process, checkpoint-bounded save, trainer-based inference.

Tiny glossary

Term	Meaning
Mode	Objective family: Max Altitude, Flight School, or incomplete Dogfight.
Track	Active scene identity used for settings, save slots, checkpoints, and RL run IDs.
Paradigm	Top-level training lifecycle strategy: evolutionary or RL.
Engine	Evolution-only population algorithm: Classic NeuroEvo or NEAT.
Brain	Callable Unity-side policy; current PPO/SAC policies are not IBrain objects.
Champion	All-time best evolutionary brain or best RL episode score metadata.
Inference	One-policy replay with no learning and no new save.

LAST LINE TO REMEMBER

The project succeeds architecturally because the aircraft and task are shared, while the learning lifecycles remain correctly different.